



Lumi Reading · Security & Privacy Evidence

T1

Security Monitoring Plan

Lumi Education Pty Ltd · ABN 45 700 349 015 · 24 July 2026

DRAFT

ST4S item: T1 (risk-based, continuous monitoring) **Version:** 0.1 DRAFT · **Date:** 2026-07-24 **Status:** Draft for review — not yet signed

1. Purpose and scope

This plan describes how Lumi Reading continuously monitors for security vulnerabilities and operational failures, triages what it finds by risk, and schedules the periodic assessments (self-managed and external) that keep the picture current. It covers the Lumi-owned estate: the Flutter app, Cloud Functions, `packages/server-ops`, the two Next.js portals, the marketing site, and the security rules — all in `lumi-ninc-au` (`australia-southeast1`). It does **not** cover the vendors' own infrastructure (Google/Firebase, SendGrid, Twilio, Apple/Google stores, Open Library/Google Books); those are governed by the vendor register and monitored only through Lumi's configuration and use of them.

Monitoring is **risk-based**: findings are ranked by severity and blast radius, recorded in a single register, and worked in priority order (§5). It has four layers — automated scanning (§3), dependency management (§4), the self-managed assessment cadence (§6), and an independent external penetration test (§7) — plus operational health monitoring (§8).

2. Roles

- Security Lead (Nic / Director):** owns this plan, the findings register, triage decisions and exceptions.
- Release approver:** enforces the per-release privacy/security gate (`docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md`).
- External tester (to be engaged):** performs the annual independent penetration test (§7).

3. Automated vulnerability scanning — per-deploy and monthly

All scanners are CI workflows in `.github/workflows/` (see EV13 §5 for detail):

Scanner	What it covers	Cadence	Mode
<code>security-review.yml</code>	Functions unit tests, Firestore rules emulator suite, production dependency audit (<code>npm audit --omit=dev --audit-level=high</code> ; <code>pnpm audit -prod --audit-level high</code>)	Weekly (Mon 02:17 UTC) + on demand	Blocking
<code>demo-readiness.yml</code>	Full regression gate incl. Firestore + Storage rules emulator suites, Flutter suite, portal typecheck/security tests	Every PR + push to <code>main</code>	Blocking

Scanner	What it covers	Cadence	Mode
<code>secret-scan.yml</code>	<code>gitleaks</code> over the complete git history (redacted)	Every push + every PR + on demand	Blocking
<code>sca-osv.yml</code>	<code>osv-scanner</code> over every lockfile (npm + pub), SARIF → Security tab	Monthly (1st, 03:17 UTC) + on demand	Report-only
<code>sast-semgrep.yml</code>	<code>semgrep</code> SAST (typescript/javascript/secrets rulesets), SARIF → Security tab	Monthly (1st, 03:23 UTC) + on demand	Report-only
<code>admin-ci.yml</code>	Admin typecheck/lint/build + feature-flag parity	Every admin PR + push to <code>main</code>	Blocking
<code>check-storage-rules.sh</code> , <code>check-csv-exports.sh</code>	Fail-closed regression guardrails — Storage-rules→Firestore-read detection; CSV/formula-injection export sinks (F-10)	Every PR + <code>firebase.json</code> predeploy	Blocking

This satisfies the "scan on every deployment and at least monthly" expectation: the dependency audit and rules/regression suites run on **every PR** (per-deploy) and again **weekly**; osv-scanner and semgrep run **monthly** and on demand. Scanner outputs land in the GitHub Security tab (SARIF) and are triaged into the findings register (§5).

Current gap (tracked): osv-scanner and semgrep are report-only with per-PR triggers disabled pending action-ref/baseline verification. The blocking dependency control today is the weekly/PR `npm/pnpm audit`. Re-enabling the per-PR SCA/SAST triggers and making them blocking is the intended end state.

4. Dependency management (SCA)

- Dependabot** (`.github/dependabot.yml`) opens weekly dependency PRs for npm (`/functions` , `/school-admin-web` , `/admin` , `/marketing-site` , root), `pub` (Flutter), and `github-actions` .
- osv-scanner** (§3) and the **weekly production** `npm/pnpm audit` (§3) cross-check the same tree.
- Open dependency risk is tracked as **SCA-01** in the register: nine transitive advisories (eight moderate, one high) under `firebase-admin` , with no clean upstream fix available (a forced upgrade breaks). Disposition: tracked, bump `firebase-admin` when a patched release lands; Dependabot + osv-scanner watch it. Patch deadlines and the patch process itself belong to the (separately drafted) EV12 Patch Management Process.

5. Risk-based triage — the findings register

The single source of truth is `docs/security/FINDINGS_REGISTER.md` : one row per finding with a severity, ST4S mapping, status (open / in-progress / confirmed-unfixed / fixed+verified / accepted-by-design), owner and decision, plus a dated change log. It aggregates findings from the self-managed assessment, the mobile (AP2/MASVS) review, `npm/pnpm audit` , osv-scanner and semgrep, de-duplicated against the closed dry-run findings (`PENTEST_DRYRUN_FINDINGS_2026-07-20.md`).

Triage rule: severity is reconciled against **current blast radius** (a control-bypass framing may be High while the live impact is Medium), and items are worked highest-first. Fixed findings must carry a regression test before being marked verified (rules fixes → `functions/test/security_poc.rules.test.js`); accepted risks must carry a written rationale (e.g. F-06 storage first-claim, F-08 SameSite CSRF). This is the T1 risk-based-triage evidence.

6. Self-managed assessment cadence (EV11 feeder)

Lumi runs a structured, source-code-assisted internal assessment against OWASP ASVS/WSTG, OWASP Top 10 (2025) + SANS CWE Top 25, and MASVS/MASTG for mobile, with all dynamic testing confined to the Firebase Emulator Suite on synthetic data (production is treated read-only). The current report is [docs/security/VULNERABILITY_ASSESSMENT_REPORT_2026-07-24.md](#) (EV11), with method in [SECURITY_ASSESSMENT_ORCHESTRATION_PLAN.md](#) and live status in [ASSESSMENT_STATUS.md](#).

Cadence:

Review	Frequency	Evidence
Automated Functions/rules/dependency review	Weekly + on demand	.github/workflows/security-review.yml
Full self-managed vulnerability assessment	At least annually, and before a major release or the ST4S full-assessment nomination	VULNERABILITY_ASSESSMENT_REPORT_*.md
Per-release privacy/security review	Every relevant PR/release; full review quarterly during beta	docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md
IAM keys/roles + deploy WIF review	Monthly + after deployment changes	IAM export + capability canaries
Vendor/subprocessor/APP 8 register	Quarterly + before new vendor use	Dated register approval
Breach tabletop	Six-monthly + after a material incident	Scenario/action record

The per-release and recurring schedule are defined authoritatively in the release-gate doc's "Recurring schedule" table; this plan references rather than duplicates it.

7. Independent external penetration test (annual)

An independent external penetration test is planned **annually** and provides the EV10 evidence.

- **Status:** OUTSTANDING. The scoping pack — [docs/security/PENETRATION_TEST_SCOPING_PACK.md](#) — is ready: it defines in-scope targets (both portals, the marketing lead forms, Firestore/Storage rules as the primary boundary, ~60 Cloud Functions, Auth/App Check, the AI comprehension pipeline, and optional mobile MASVS), out-of-scope items, the no-real-child-data constraint, a grey-box approach (rules provided to the tester), OWASP ASVS/WSTG + MASVS methodology, required deliverables (executive summary, per-finding ratings, a redacted shareable report, and a re-test), and the rules-of-engagement to be finalised in the SOW.
- **Next steps** (from the pack §9): send the RFQ ([PENETRATION_TEST_RFQ_EMAIL.md](#)) to CREST-accredited AU firms, stand up the staging project with seeded role accounts, sign the SOW, schedule, then remediate → re-test → file the redacted report as EV10.

- Until the external test lands, the self-managed report (§6) stands as the **interim** vulnerability-assessment evidence (EV11).

8. Operational health monitoring

Security depends on the backend actually running, so operational health is monitored continuously:

- Cron heartbeats + storage metrics** are written to the `opsMetrics` collection (deny-all to clients); the admin dashboard polls them, and threshold alert policies are attached to both security email channels.
- `scripts/audit-function-health.sh` is a read-only, one-shot audit of the Gen2 Functions estate that catches the silent-failure classes: per-service invoker-403s, missing `run.invoker` bindings on trigger/scheduler-backed services, dropped events (maxInstances saturation), scheduler jobs whose last attempt failed, and cron heartbeats that are missing/errored/stale beyond their cadence. It **auto-runs** via a committed PostToolUse hook after a functions deploy or a gcloud IAM change, and is run by hand after infra changes made outside the repo.
- The most recent operations audit (`docs/security/OPERATIONS_HEALTH_AUDIT_2026-07-17.md`) recorded backups (7-day PITR + deletion protection, restore drill matched production counts), dedicated runtime identities, 13 enabled alert policies on both channels, and a repaired Scheduler identity drift — a concrete instance of this monitoring catching a live 403 fault.

9. Consolidated cadence

Activity	Frequency	Owner	Evidence
Regression + rules + dependency-audit CI	Every PR / push to <code>main</code>	CI	EV13 §5 workflows
Secret scan (full history)	Every push / PR	CI	<code>secret-scan.yml</code>
Weekly automated security review	Weekly	CI / Security Lead	<code>security-review.yml</code>
SCA (osv-scanner) + SAST (semgrep)	Monthly + on demand	Security Lead	<code>sca-osv.yml</code> , <code>sast-semgrep.yml</code>
Dependency-update PRs	Weekly	Dependabot / Security Lead	<code>dependabot.yml</code>
Function-health audit	After each functions deploy / IAM change + on demand	Security Lead	<code>scripts/audit-function-health.sh</code>
Self-managed vulnerability assessment	Annually + pre-major-release	Security Lead	<code>VULNERABILITY_ASSESSMENT_REPORT_*.md</code>
External penetration test	Annually	External tester	<code>PENETRATION_TEST_SCOPING_PACK.md</code> → EV10 (pending)
IAM / vendor / PIA / tabletop	Monthly / quarterly / six-monthly	Security Lead	release-gate schedule table

10. Known gaps (for the reviewer)

- **EV10 external test is not yet done** — the scoping pack is ready but no firm is engaged. This is the single largest outstanding monitoring input; §7 tracks it.
- **SCA/SAST are report-only and monthly**, not per-PR blocking (§3). Confirm the intended end state and whether monthly is sufficient for T1 in the interim.
- **Alert-delivery receipt** (that a synthetic alert reaches both inboxes) and **named incident roles** are open human items in the operations audit — verify these are closed.
- **Cadence ownership**: the plan names the Security Lead as owner throughout; confirm the named human and that the annual/monthly reviews are actually calendared, not just documented.